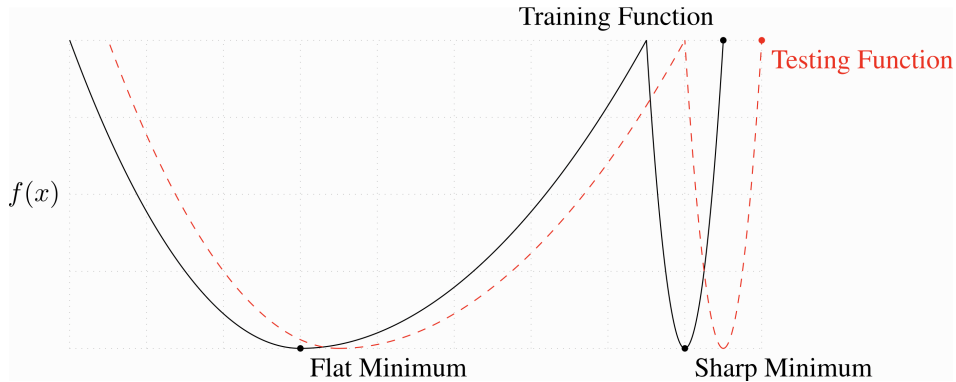


# Advanced stochastic methods. Variance reduction

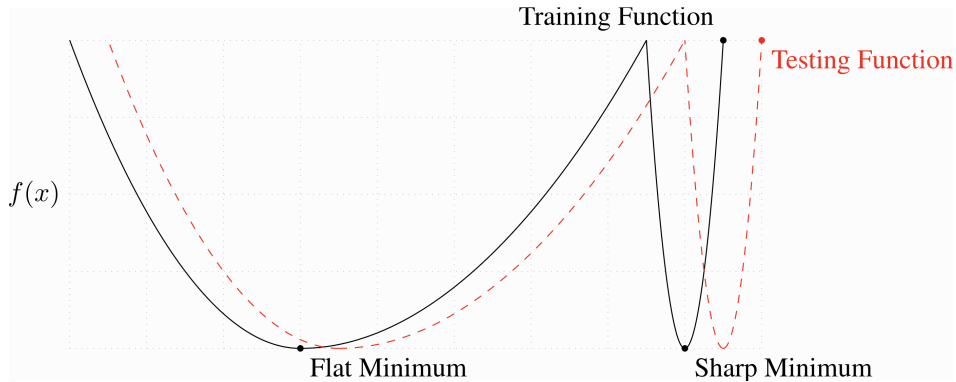
## Seminar

Optimization for ML. Faculty of Computer Science. HSE University

# Flat Minimum vs Sharp Minimum



# Flat Minimum vs Sharp Minimum



## i Question

What's wrong with Sharp Minimum?

# Sharpness-Aware Minimization<sup>1</sup>

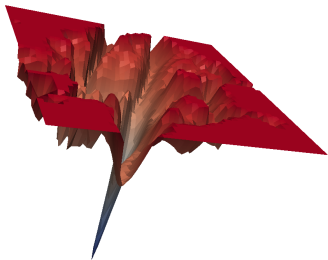


Figure 1: A sharp minimum to which a ResNet trained with SGD converged.

# Sharpness-Aware Minimization<sup>1</sup>

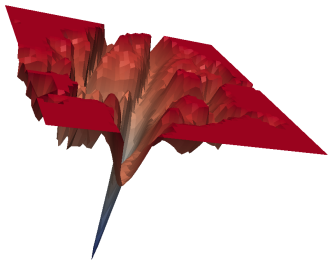


Figure 1: A sharp minimum to which a ResNet trained with SGD converged.

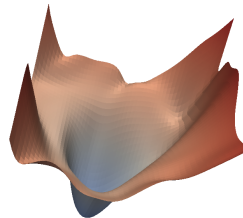


Figure 2: A wide minimum to which the same ResNet trained with SAM converged.

---

<sup>1</sup>Foret, Pierre, et al. "Sharpness-aware minimization for efficiently improving generalization." (2020).

# Sharpness-Aware Minimization<sup>1</sup>

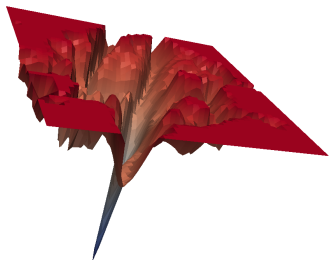


Figure 1: A sharp minimum to which a ResNet trained with SGD converged.

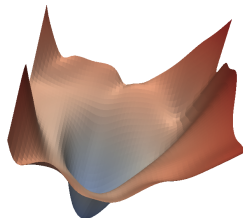


Figure 2: A wide minimum to which the same ResNet trained with SAM converged.

Sharpness-Aware Minimization (SAM) is a procedure that aims to improve model generalization by simultaneously minimizing loss value and **loss sharpness**.

<sup>1</sup>Foret, Pierre, et al. "Sharpness-aware minimization for efficiently improving generalization." (2020).

# Learning setup

The training dataset drawn *i.i.d.* from a distribution  $D$ :

$$S = \{(x_i, y_i)\}_{i=1}^n,$$

where  $x_i$  – feature vector and  $y_i$  – label.

## Learning setup

The training dataset drawn *i.i.d.* from a distribution  $D$ :

$$S = \{(x_i, y_i)\}_{i=1}^n,$$

where  $x_i$  – feature vector and  $y_i$  – label.

The training set loss:

$$L_S = \frac{1}{n} \sum_{i=1}^n l(\mathbf{w}, x_i, y_i),$$

where  $l$  – per-data-point loss function,  $\mathbf{w}$  – parameters.

# Learning setup

The training dataset drawn *i.i.d.* from a distribution  $D$ :

$$S = \{(x_i, y_i)\}_{i=1}^n,$$

where  $x_i$  – feature vector and  $y_i$  – label.

The training set loss:

$$L_S = \frac{1}{n} \sum_{i=1}^n l(\mathbf{w}, x_i, y_i),$$

where  $l$  – per-data-point loss function,  $\mathbf{w}$  – parameters.

The population loss:

$$L_D = \mathbb{E}_{(x,y)}[l(\mathbf{w}, x, y)]$$

# What is sharpness?

## i Theorem

For any  $\rho > 0$ , with high probability over training set  $S$  generated from distribution  $D$ ,

$$L_D(\mathbf{w}) \leq \max_{\|\epsilon\|_2 \leq \rho} L_S(\mathbf{w} + \epsilon) + h\left(\|\mathbf{w}\|_2^2 / \rho^2\right),$$

where  $h : \mathbb{R}_+ \rightarrow \mathbb{R}_+$  is a strictly increasing function (under some technical conditions on  $L_D(\mathbf{w})$ ).

# What is sharpness?

## i Theorem

For any  $\rho > 0$ , with high probability over training set  $S$  generated from distribution  $D$ ,

$$L_D(\mathbf{w}) \leq \max_{\|\epsilon\|_2 \leq \rho} L_S(\mathbf{w} + \epsilon) + h\left(\|\mathbf{w}\|_2^2 / \rho^2\right),$$

where  $h : \mathbb{R}_+ \rightarrow \mathbb{R}_+$  is a strictly increasing function (under some technical conditions on  $L_D(\mathbf{w})$ ).

Adding and subtracting  $L_S(\mathbf{w})$ :

$$\left[ \max_{\|\epsilon\|_2 \leq \rho} L_S(\mathbf{w} + \epsilon) - L_S(\mathbf{w}) \right] + L_S(\mathbf{w}) + h\left(\|\mathbf{w}\|_2^2 / \rho^2\right)$$

The term in square brackets captures the **sharpness** of  $L_S$  at  $\mathbf{w}$  by measuring how quickly the training loss can be increased by moving from  $\mathbf{w}$  to a nearby parameter value.

# Sharpness-Aware Minimization

The function  $h$  is removed in favor of a simpler constant  $\lambda$ . The authors propose selecting parameter values by solving the following Sharpness-Aware Minimization (SAM) problem:

$$\min_{\mathbf{w}} L_S^{SAM}(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2 \quad \text{where} \quad L_S^{SAM}(\mathbf{w}) \triangleq \max_{\|\epsilon\|_p \leq \rho} L_S(\mathbf{w} + \epsilon),$$

with  $\rho \geq 0$  as hyperparameter and  $p$  in  $[1, \infty]$  (a little generalization, though  $p = 2$  is empirically the best choice).

## How to minimize $L_S^{SAM}$ ?

In order to minimize  $L_S^{SAM}$  an efficient approximation of its gradient is used. A first step is to consider the first-order Taylor expansion of  $L_S(\mathbf{w} + \boldsymbol{\epsilon})$ :

$$\boldsymbol{\epsilon}^*(\mathbf{w}) \triangleq \arg \max_{\|\boldsymbol{\epsilon}\|_p \leq \rho} L_S(\mathbf{w} + \boldsymbol{\epsilon}) \approx \arg \max_{\|\boldsymbol{\epsilon}\|_p \leq \rho} L_S(\mathbf{w}) + \boldsymbol{\epsilon}^T \nabla_{\mathbf{w}} L_S(\mathbf{w}) = \arg \max_{\|\boldsymbol{\epsilon}\|_p \leq \rho} \boldsymbol{\epsilon}^T \nabla_{\mathbf{w}} L_S(\mathbf{w}).$$

## How to minimize $L_S^{SAM}$ ?

In order to minimize  $L_S^{SAM}$  an efficient approximation of its gradient is used. A first step is to consider the first-order Taylor expansion of  $L_S(\mathbf{w} + \boldsymbol{\epsilon})$ :

$$\boldsymbol{\epsilon}^*(\mathbf{w}) \triangleq \arg \max_{\|\boldsymbol{\epsilon}\|_p \leq \rho} L_S(\mathbf{w} + \boldsymbol{\epsilon}) \approx \arg \max_{\|\boldsymbol{\epsilon}\|_p \leq \rho} L_S(\mathbf{w}) + \boldsymbol{\epsilon}^T \nabla_{\mathbf{w}} L_S(\mathbf{w}) = \arg \max_{\|\boldsymbol{\epsilon}\|_p \leq \rho} \boldsymbol{\epsilon}^T \nabla_{\mathbf{w}} L_S(\mathbf{w}).$$

The last expression is just the argmax of the dot product of the vectors  $\boldsymbol{\epsilon}$  and  $\nabla_{\mathbf{w}} L_S(\mathbf{w})$ , and it is well known which is the argument that maximizes it:

$$\hat{\boldsymbol{\epsilon}}(\mathbf{w}) = \rho \operatorname{sign}(\nabla_{\mathbf{w}} L_S(\mathbf{w})) |\nabla_{\mathbf{w}} L_S(\mathbf{w})|^{q-1} / \left( \|\nabla_{\mathbf{w}} L_S(\mathbf{w})\|_q^q \right)^{1/p},$$

where  $1/p + 1/q = 1$ .

## How to minimize $L_S^{SAM}$ ?

In order to minimize  $L_S^{SAM}$  an efficient approximation of its gradient is used. A first step is to consider the first-order Taylor expansion of  $L_S(\mathbf{w} + \boldsymbol{\epsilon})$ :

$$\boldsymbol{\epsilon}^*(\mathbf{w}) \triangleq \arg \max_{\|\boldsymbol{\epsilon}\|_p \leq \rho} L_S(\mathbf{w} + \boldsymbol{\epsilon}) \approx \arg \max_{\|\boldsymbol{\epsilon}\|_p \leq \rho} L_S(\mathbf{w}) + \boldsymbol{\epsilon}^T \nabla_{\mathbf{w}} L_S(\mathbf{w}) = \arg \max_{\|\boldsymbol{\epsilon}\|_p \leq \rho} \boldsymbol{\epsilon}^T \nabla_{\mathbf{w}} L_S(\mathbf{w}).$$

The last expression is just the argmax of the dot product of the vectors  $\boldsymbol{\epsilon}$  and  $\nabla_{\mathbf{w}} L_S(\mathbf{w})$ , and it is well known which is the argument that maximizes it:

$$\hat{\boldsymbol{\epsilon}}(\mathbf{w}) = \rho \operatorname{sign}(\nabla_{\mathbf{w}} L_S(\mathbf{w})) |\nabla_{\mathbf{w}} L_S(\mathbf{w})|^{q-1} / \left( \|\nabla_{\mathbf{w}} L_S(\mathbf{w})\|_q^q \right)^{1/p},$$

where  $1/p + 1/q = 1$ .

Thus

$$\begin{aligned} \nabla_{\mathbf{w}} L_S^{SAM}(\mathbf{w}) &\approx \nabla_{\mathbf{w}} L_S(\mathbf{w} + \hat{\boldsymbol{\epsilon}}(\mathbf{w})) = \left. \frac{d(\mathbf{w} + \hat{\boldsymbol{\epsilon}}(\mathbf{w}))}{d\mathbf{w}} \nabla_{\mathbf{w}} L_S(\mathbf{w}) \right|_{\mathbf{w} + \hat{\boldsymbol{\epsilon}}(\mathbf{w})} \\ &= \nabla_{\mathbf{w}} L_S(\mathbf{w})|_{\mathbf{w} + \hat{\boldsymbol{\epsilon}}(\mathbf{w})} + \left. \frac{d\hat{\boldsymbol{\epsilon}}(\mathbf{w})}{d\mathbf{w}} \nabla_{\mathbf{w}} L_S(\mathbf{w}) \right|_{\mathbf{w} + \hat{\boldsymbol{\epsilon}}(\mathbf{w})} \end{aligned}$$

## Sharpness-Aware Minimization

Modern frameworks can easily compute the preceding approximation. However, to speed up the computation, second-order terms can be dropped obtaining:

$$\nabla_w L_S^{SAM}(w) \approx \nabla_w L_S(w) \Big|_{w + \hat{\epsilon}(w)}$$

# Sharpness-Aware Minimization

Modern frameworks can easily compute the preceding approximation. However, to speed up the computation, second-order terms can be dropped obtaining:

$$\nabla_w L_S^{SAM}(w) \approx \nabla_w L_S(w) \Big|_{w+\hat{\epsilon}(w)}$$

**Input:** Training set  $\mathcal{S} \triangleq \cup_{i=1}^n \{(\mathbf{x}_i, \mathbf{y}_i)\}$ , Loss function  $l : \mathcal{W} \times \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}_+$ , Batch size  $b$ , Step size  $\eta > 0$ , Neighborhood size  $\rho > 0$ .

**Output:** Model trained with SAM

Initialize weights  $w_0, t = 0$ ;

**while not converged do**

    Sample batch  $\mathcal{B} = \{(\mathbf{x}_1, \mathbf{y}_1), \dots (\mathbf{x}_b, \mathbf{y}_b)\}$ ;

    Compute gradient  $\nabla_w L_{\mathcal{B}}(w)$  of the batch's training loss;

    Compute  $\hat{\epsilon}(w)$  per equation 2;

    Compute gradient approximation for the SAM objective

        (equation 3):  $g = \nabla_w L_{\mathcal{B}}(w) \Big|_{w+\hat{\epsilon}(w)}$ ;

    Update weights:  $w_{t+1} = w_t - \eta g$ ;

$t = t + 1$ ;

**end**

**return**  $w_t$

**Algorithm 1:** SAM algorithm

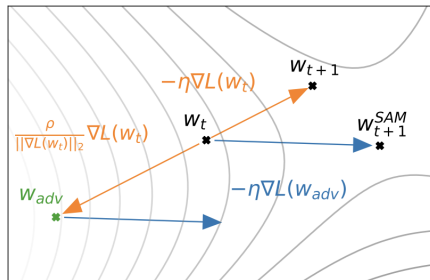


Figure 2: Schematic of the SAM parameter update.

## SAM results

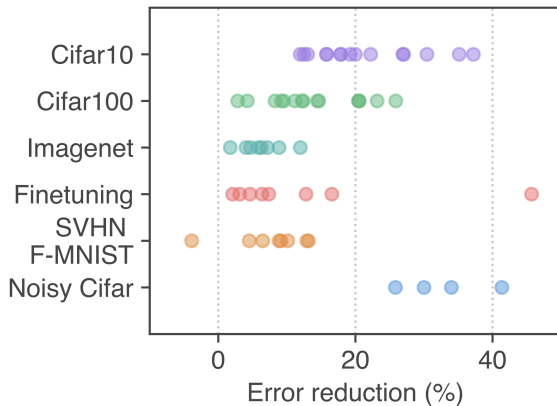


Figure 4: Error rate reduction obtained by switching to SAM. Each point is a different dataset / model / data augmentation.

# Mode Connectivity<sup>2</sup>

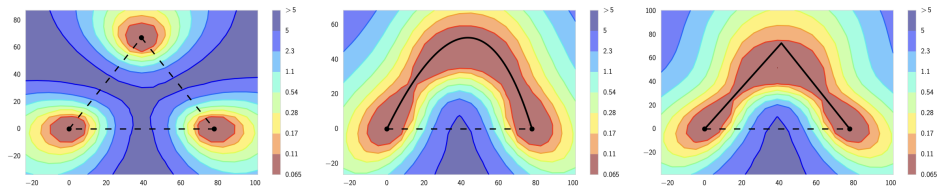


Figure 5: The  $l_2$ -regularized cross-entropy train loss surface of a ResNet-164 on CIFAR-100, as a function of network weights in a two-dimensional subspace. In each panel, the horizontal axis is fixed and is attached to the optima of two independently trained networks. The vertical axis changes between panels as we change planes (defined in the main text). Left: Three optima for independently trained networks. Middle and Right: A quadratic Bezier curve, and a polygonal chain with one bend, connecting the lower two optima on the left panel along a path of near-constant loss. Notice that in each panel a direct linear path between each mode would incur high loss.

<sup>2</sup>Garipov, Timur, et al. "Loss surfaces, mode connectivity, and fast ensembling of dnns." Advances in neural information processing systems 31 (2018).

# Curve-Finding Procedure

- Weights of pretrained networks:

$$\hat{w}_1, \hat{w}_2 \in \mathbb{R}^{|\text{net}|}$$

- Define parametric curve:  $\phi_\theta(\cdot) : [0, 1] \rightarrow \mathbb{R}^{|\text{net}|}$

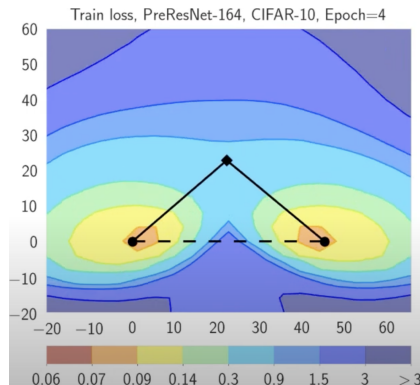
$$\phi_\theta(0) = \hat{w}_1, \quad \phi_\theta(1) = \hat{w}_2$$

- DNN loss function:

$$\mathcal{L}(w)$$

- Minimize averaged loss w.r.t.  $\theta$ :

$$\underset{\theta}{\text{minimize}} \quad \ell(\theta) = \int_0^1 \mathcal{L}(\phi_\theta(t)) dt = \mathbb{E}_{t \sim U(0,1)} \mathcal{L}(\phi_\theta(t))$$



# Curve-Finding Procedure

- Weights of pretrained networks:

$$\hat{w}_1, \hat{w}_2 \in \mathbb{R}^{|\text{net}|}$$

- Define parametric curve:  $\phi_\theta(\cdot) : [0, 1] \rightarrow \mathbb{R}^{|\text{net}|}$

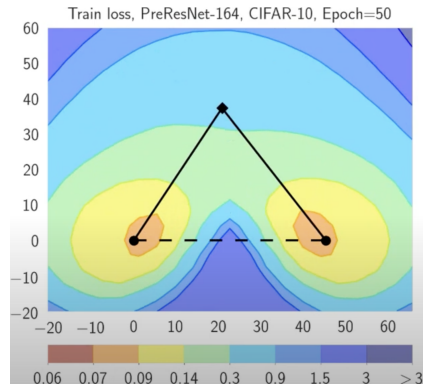
$$\phi_\theta(0) = \hat{w}_1, \quad \phi_\theta(1) = \hat{w}_2$$

- DNN loss function:

$$\mathcal{L}(w)$$

- Minimize averaged loss w.r.t.  $\theta$ :

$$\underset{\theta}{\text{minimize}} \quad \ell(\theta) = \int_0^1 \mathcal{L}(\phi_\theta(t)) dt = \mathbb{E}_{t \sim U(0,1)} \mathcal{L}(\phi_\theta(t))$$



# Curve-Finding Procedure

- Weights of pretrained networks:

$$\hat{w}_1, \hat{w}_2 \in \mathbb{R}^{|\text{net}|}$$

- Define parametric curve:  $\phi_\theta(\cdot) : [0, 1] \rightarrow \mathbb{R}^{|\text{net}|}$

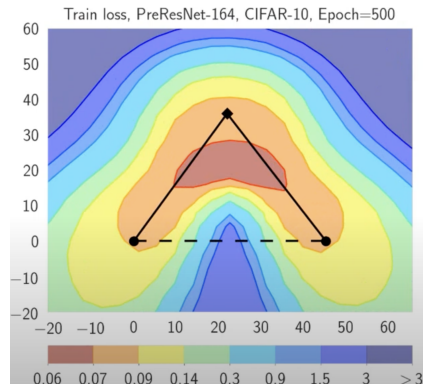
$$\phi_\theta(0) = \hat{w}_1, \quad \phi_\theta(1) = \hat{w}_2$$

- DNN loss function:

$$\mathcal{L}(w)$$

- Minimize averaged loss w.r.t.  $\theta$ :

$$\underset{\theta}{\text{minimize}} \quad \ell(\theta) = \int_0^1 \mathcal{L}(\phi_\theta(t)) dt = \mathbb{E}_{t \sim U(0,1)} \mathcal{L}(\phi_\theta(t))$$



# Grokking<sup>3</sup>

- After achieving zero train loss the weights continue evolving in a kind of random walk manner
- It is possible that they slowly drift to a wider minima
- Recently discovered grokking effect confirms this hypo

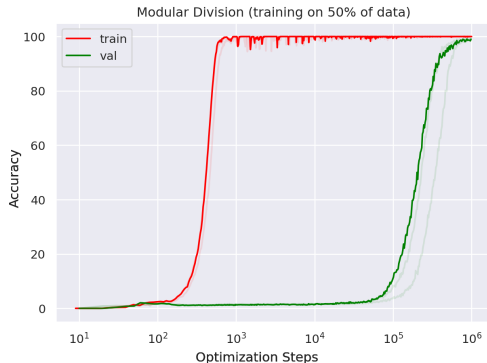


Figure 6: Grokking: A dramatic example of generalization far after overfitting on an algorithmic dataset.

<sup>3</sup>Power, Alethea, et al. "Grokking: Generalization beyond overfitting on small algorithmic datasets." (2022).

## Double Descent<sup>4</sup>

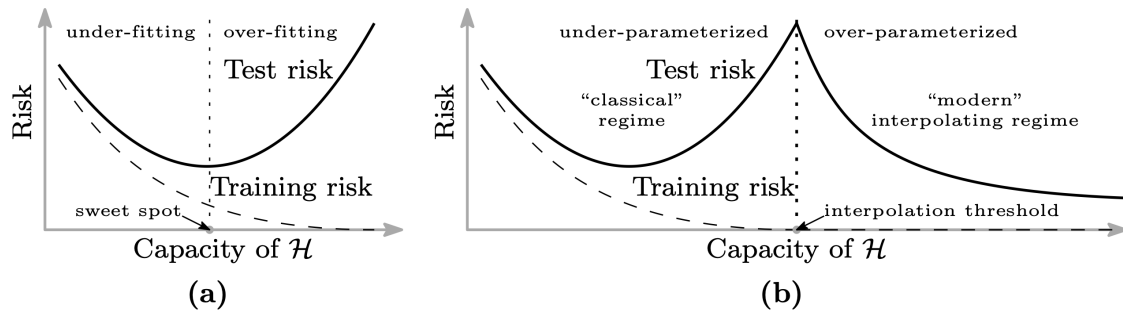


Figure 7: Curves for training risk (dashed line) and test risk (solid line). (a) The classical U-shaped risk curve arising from the bias-variance trade-off. (b) The double descent risk curve, which incorporates the U-shaped risk curve (i.e., the “classical” regime) together with the observed behavior from using high capacity function classes (i.e., the “modern” interpolating regime), separated by the interpolation threshold. The predictors to the right of the interpolation threshold have zero training risk.

<sup>4</sup>Belkin, Mikhail, et al. “Reconciling modern machine-learning practice and the classical bias–variance trade-off.” (2019)

## A simple Grokking implementation and a useful library

- Simple PyTorch Implementation of Grokking you can find here ♣
- PyTorch-based library for approximating and visualisation neural network loss functions, and other related metrics, in low-dimensional subspaces of the model's parameter space ♣